Insertion Sort

Array → 5, 3, 4, 1, 2 [Partially sorting the array]

[For every index, put that index element at the correct index of LHS.]

i.e., 1st pass → $i = 0$

sorted ← 5, 3, 4, 1, 2 → 3, 5, 4, 1, 2

2nd pass → $i = 1$

sorted ← 3, 5, 4, 1, 2 → 3, 4, 5, 1, 2

3rd pass → $i = 2$

sorted ← 3, 4, 5, 1, 2 → 1, 3, 4, 5, 2

4th pass → $i = 3$

sorted ← 1, 3, 4, 5, 2 → $\boxed{1, 2, 3, 4, 5}$ → Array is sorted

- **TIME COMPLEXITY:**

① **WORST CASE:** $O(n^2)$

(Descending sorted)

② **BEST CASE:** $O(n)$

(Already sorted)

* Why should you use insertion sort?

* Adaptive:

Steps get reduced if array is sorted [i.e. no. of swaps are reduced (as compared to bubble sort)]

* Stable sorting algorithm

* Used for smaller values of n:

→ Works good when array is partially sorted.

→ It takes part in hybrid sorting algorithm.